

Build a Role-Based Document-Aware Chatbot

Background

In a banking environment, employees frequently need to refer to internal documents—such as Standard Operating Procedures (SOPs), compliance manuals, audit guidelines, and client onboarding checklists—to answer operational queries. This manual process is time-consuming and prone to delays.

To improve efficiency, the bank wants to develop a chatbot that can automatically respond to employee queries by referencing pre-uploaded documents. Certain users (e.g., team leads or compliance officers) should have access to upload new documents, while others should only be able to query the bot. Role-based access and secure login are essential.

Objective

Design and implement a chatbot that:

1. Responds to user queries by searching through pre-uploaded documents.
2. Allows authorized users to upload new documents.
3. Enforces role-based access control (RBAC) via login credentials.

Functional Requirements

Step 1: User Login

- Implement a secure login system with username and password.
- Define roles:
 - Admin: Can upload, delete, and manage documents.
 - User: Can query the chatbot but cannot upload documents.

Step 2: Document Upload (Admin Only)

- Accept PDF, DOCX, PPTX, XLSX, images/scanned documents
- Store documents in a searchable format (e.g., indexed text or vector embeddings).
- Extract and preprocess text for efficient querying.

Step 3: Chatbot Interface

- Users can ask questions in natural language.
- The chatbot searches the document database and returns relevant answers.
- Responses must cite the source document and section.

Step 4: Role-Based Access Control

- Restrict document upload and management to Admins.
- Allow only authenticated users to interact with the chatbot.
- Log all user actions for audit purposes.

Technical Expectations

- Students are free to use the technology as per their best judgement
- Mandatory: All code must be documented with docstrings and inline comments.

Milestones & Timeline

#	Milestone	Description	Timeline
1	Requirements & Design	Define chatbot flow, roles, and document handling	Week 1
2	Login & Role System	Implement user authentication and RBAC	Week 2
3	Document Upload Module	Admin-only upload and parsing	Week 3
4	Chatbot Engine	Build query interface and document search logic	Week 4
5	Response Generation	Format answers with citations	Week 5
6	Documentation & Testing	Add docstrings, comments, and test cases	Week 6
7	Final Demo & Submission	Present working chatbot with sample documents	Week 7

Qualifying Criteria

#	Criteria	Description
1	Functionality	Chatbot must respond accurately using document content
2	Role Enforcement	Upload access must be restricted to Admins
3	Document Coverage	Must support at least 4 document types (PDF, DOCX, PPTX, XLSX, images and scanned documents)
4	User Experience	Interface should be intuitive and responsive
5	Error Handling	Tool must handle invalid queries and login failures gracefully
6	Code Quality	Code must be modular, readable, and well-documented
7	Documentation	Every function/class must include docstrings and inline comments
8	Accuracy Benchmark	Chatbot must achieve ≥85% accuracy in retrieving correct responses from documents (based on test queries)
9	Demo	A short video or live walkthrough must be provided
10	Sample Documents	Include at least three sample documents for testing